

```
# =====
# CCS2213 - Machine Learning | Project #7
# Employee Productivity Analysis and Attrition Prediction
# =====
# STEP 1 - DATA PREPROCESSING
# Deliverables: #2 (EDA), #3 (Missing Values), #4 (Scaling),
#               #5 (Encoding), #6 (Feature Engineering + Split)
# =====
```

```
# — 0. INSTALL & IMPORT —————
# !pip install pandas numpy matplotlib seaborn scikit-learn --quiet
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
```

```
import warnings
warnings.filterwarnings('ignore')
```

```
# Plot style
sns.set_theme(style="whitegrid", palette="Set2")
plt.rcParams['figure.figsize'] = (10, 5)
```

```
print("=" * 60)
print("STEP 1 - DATA PREPROCESSING")
print("=" * 60)
```

```
=====
STEP 1 - DATA PREPROCESSING
=====
```

```
# — 1. LOAD DATASET —————
# Download from Kaggle:
# https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset
# Upload to Colab or place in same directory
```

```
df = pd.read_csv('WA_Fn-UseC_-HR-Employee-Attrition.csv')

print(f"\n[1] Dataset loaded successfully.")
print(f"    Shape: {df.shape[0]} rows x {df.shape[1]} columns")
```

```
[1] Dataset loaded successfully.
    Shape: 1470 rows x 35 columns
```

```
# — 2. INITIAL EDA —————
print("\n" + "-" * 60)
print("[2] INITIAL EXPLORATORY DATA ANALYSIS")
print("-" * 60)
```

```
print("\n>> Data Types & Non-Null Counts:")
print(df.info())
```

```
print("\n>> First 5 Rows:")
print(df.head())
```

```
print("\n>> Descriptive Statistics (Numerical Columns):")
print(df.describe().round(2))
```

```
print("\n>> Target Variable Distribution:")
attrition_counts = df['Attrition'].value_counts()
attrition_pct    = df['Attrition'].value_counts(normalize=True) * 100
print(attrition_counts)
print(f"\n    No (0): {attrition_pct['No']:.1f}%")
print(f"    Yes (1): {attrition_pct['Yes']:.1f}%")
print("\n    NOTE: Dataset is imbalanced (~84% No, ~16% Yes).")
print("        class_weight='balanced' will be used in all models.")
```

```
# Plot target distribution
fig, ax = plt.subplots(figsize=(6, 4))
attrition_counts.plot(kind='bar', color=['#2E7D32', '#C62828'], edgecolor='white', ax=ax)
ax.set_title('Target Variable Distribution - Attrition', fontsize=13, fontweight='bold')
ax.set_xlabel('Attrition')
ax.set_ylabel('Count')
ax.set_xticklabels(['No', 'Yes'], rotation=0)
```

```
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width() / 2, p.get_height()),
               ha='center', va='bottom', fontsize=11)
plt.tight_layout()
plt.savefig('fig_target_distribution.png', dpi=150)
plt.show()
print("    [Saved] fig_target_distribution.png")

# Categorical columns overview
cat_cols = df.select_dtypes(include='object').columns.tolist()
num_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
print(f"\n>> Categorical columns ({len(cat_cols)}): {cat_cols}")
print(f">> Numerical columns ({len(num_cols)}): {num_cols}")
```


[2] INITIAL EXPLORATORY DATA ANALYSIS

>> Data Types & Non-Null Counts:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1470 entries, 0 to 1469

Data columns (total 35 columns):

#	Column	Non-Null Count	Dtype
0	Age	1470 non-null	int64
1	Attrition	1470 non-null	object
2	BusinessTravel	1470 non-null	object
3	DailyRate	1470 non-null	int64
4	Department	1470 non-null	object
5	DistanceFromHome	1470 non-null	int64
6	Education	1470 non-null	int64
7	EducationField	1470 non-null	object
8	EmployeeCount	1470 non-null	int64
9	EmployeeNumber	1470 non-null	int64
10	EnvironmentSatisfaction	1470 non-null	int64
11	Gender	1470 non-null	object
12	HourlyRate	1470 non-null	int64
13	JobInvolvement	1470 non-null	int64
14	JobLevel	1470 non-null	int64
15	JobRole	1470 non-null	object
16	JobSatisfaction	1470 non-null	int64
17	MaritalStatus	1470 non-null	object
18	MonthlyIncome	1470 non-null	int64
19	MonthlyRate	1470 non-null	int64
20	NumCompaniesWorked	1470 non-null	int64
21	Over18	1470 non-null	object
22	OverTime	1470 non-null	object
23	PercentSalaryHike	1470 non-null	int64
24	PerformanceRating	1470 non-null	int64
25	RelationshipSatisfaction	1470 non-null	int64
26	StandardHours	1470 non-null	int64
27	StockOptionLevel	1470 non-null	int64
28	TotalWorkingYears	1470 non-null	int64
29	TrainingTimesLastYear	1470 non-null	int64
30	WorkLifeBalance	1470 non-null	int64
31	YearsAtCompany	1470 non-null	int64
32	YearsInCurrentRole	1470 non-null	int64
33	YearsSinceLastPromotion	1470 non-null	int64
34	YearsWithCurrManager	1470 non-null	int64

dtypes: int64(26), object(9)

memory usage: 402.1+ KB

None

>> First 5 Rows:

	Age	Attrition	BusinessTravel	DailyRate	Department	\
0	41	Yes	Travel_Rarely	1102		Sales
1	49	No	Travel_Frequently	279	Research & Development	
2	37	Yes	Travel_Rarely	1373	Research & Development	
3	33	No	Travel_Frequently	1392	Research & Development	
4	27	No	Travel_Rarely	591	Research & Development	

	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber	\
0	1	2	Life Sciences	1		1
1	8	1	Life Sciences	1		2
2	2	2	Other	1		4
3	3	4	Life Sciences	1		5
4	2	1	Medical	1		7

	RelationshipSatisfaction	StandardHours	StockOptionLevel	\
0	...	1	80	0
1	...	4	80	1
2	...	2	80	0
3	...	3	80	0
4	...	4	80	1

	TotalWorkingYears	TrainingTimesLastYear	WorkLifeBalance	YearsAtCompany	\
0	8	0	1		6
1	10	3	3		10
2	7	3	3		0
3	8	3	3		8
4	6	3	3		2

	YearsInCurrentRole	YearsSinceLastPromotion	YearsWithCurrManager
0	4	0	5
1	7	1	7
2	0	0	0
3	7	3	0
4	2	2	2

[5 rows x 35 columns]

>> Descriptive Statistics (Numerical Columns):

Age DailyRate DistanceFromHome Education EmployeeCount \

count	1470.00	1470.00	1470.00	1470.00	1470.00
mean	36.92	802.49	9.19	2.91	1.0
std	9.14	403.51	8.11	1.02	0.0
min	18.00	102.00	1.00	1.00	1.0
25%	30.00	465.00	2.00	2.00	1.0
50%	36.00	802.00	7.00	3.00	1.0
75%	43.00	1157.00	14.00	4.00	1.0
max	60.00	1499.00	29.00	5.00	1.0

	EmployeeNumber	EnvironmentSatisfaction	HourlyRate	JobInvolvement	\
count	1470.00	1470.00	1470.00	1470.00	
mean	1024.87	2.72	65.89	2.73	
std	602.02	1.09	20.33	0.71	
min	1.00	1.00	30.00	1.00	
25%	491.25	2.00	48.00	2.00	
50%	1020.50	3.00	66.00	3.00	
75%	1555.75	4.00	83.75	3.00	
max	2068.00	4.00	100.00	4.00	

	JobLevel	...	RelationshipSatisfaction	StandardHours	\
count	1470.00	...	1470.00	1470.00	
mean	2.06	...	2.71	80.0	
std	1.11	...	1.08	0.0	
min	1.00	...	1.00	80.0	
25%	1.00	...	2.00	80.0	
50%	2.00	...	3.00	80.0	
75%	3.00	...	4.00	80.0	
max	5.00	...	4.00	80.0	

	StockOptionLevel	TotalWorkingYears	TrainingTimesLastYear	\
count	1470.00	1470.00	1470.00	
mean	0.79	11.28	2.80	
std	0.85	7.78	1.29	
min	0.00	0.00	0.00	
25%	0.00	6.00	2.00	
50%	1.00	10.00	3.00	
75%	1.00	15.00	3.00	
max	3.00	40.00	6.00	

	WorkLifeBalance	YearsAtCompany	YearsInCurrentRole	\
count	1470.00	1470.00	1470.00	
mean	2.76	7.01	4.23	
std	0.71	6.13	3.62	
min	1.00	0.00	0.00	
25%	2.00	3.00	2.00	
50%	3.00	5.00	3.00	
75%	3.00	9.00	7.00	
max	4.00	40.00	18.00	

	YearsSinceLastPromotion	YearsWithCurrManager
count	1470.00	1470.00
mean	2.19	4.12
std	3.22	3.57
min	0.00	0.00
25%	0.00	2.00
50%	1.00	3.00
75%	3.00	7.00
max	15.00	17.00

[8 rows x 26 columns]

>> Target Variable Distribution:

Attrition

No 1233

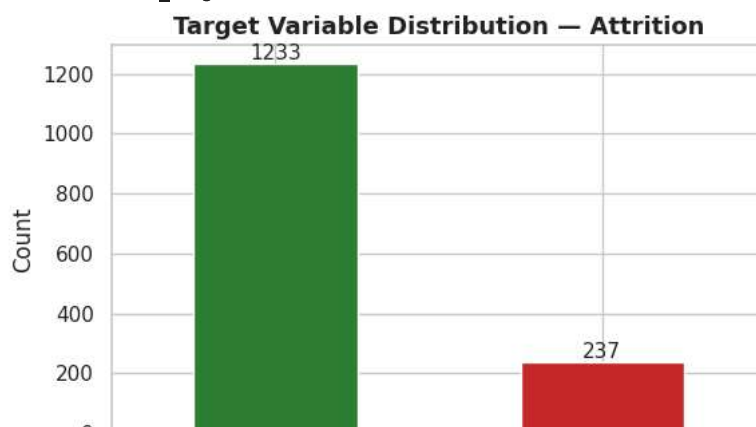
Yes 237

Name: count, dtype: int64

No (0): 83.9%

Yes (1): 16.1%

NOTE: Dataset is imbalanced (~84% No, ~16% Yes).
class_weight='balanced' will be used in all models.



```
# — 3. MISSING VALUE ANALYSIS —————
print("\n" + "-" * 60)
print("[3] MISSING VALUE ANALYSIS")
print("-" * 60)

missing = df.isnull().sum()
missing_pct = (missing / len(df)) * 100
missing_df = pd.DataFrame({'Missing Count': missing, 'Missing %': missing_pct})
missing_df = missing_df[missing_df['Missing Count'] > 0]

if missing_df.empty:
    print("\n    No missing values found. Dataset is clean.")
    print("    No imputation or removal required.")
else:
    print("\n    Columns with missing values:")
    print(missing_df)
```

[3] MISSING VALUE ANALYSIS

```
No missing values found. Dataset is clean.
No imputation or removal required.
```

```
# — 4. FEATURE ENGINEERING —————
print("\n" + "-" * 60)
print("[4] FEATURE ENGINEERING")
print("-" * 60)

# Drop constant / non-informative columns
cols_to_drop = ['EmployeeCount', 'StandardHours', 'Over18', 'EmployeeNumber']
df.drop(columns=cols_to_drop, inplace=True)
print(f"\n    Dropped non-informative columns: {cols_to_drop}")
print(f"    Reason: These columns have constant values and provide no")
print(f"            predictive signal for the model.")
print(f"\n    New shape: {df.shape[0]} rows × {df.shape[1]} columns")
```

[4] FEATURE ENGINEERING

```
Dropped non-informative columns: ['EmployeeCount', 'StandardHours', 'Over18', 'EmployeeNumber']
Reason: These columns have constant values and provide no
        predictive signal for the model.

New shape: 1470 rows × 31 columns
```

```
# — 5. ENCODE CATEGORICAL VARIABLES —————
print("\n" + "-" * 60)
print("[5] ENCODING CATEGORICAL VARIABLES")
print("-" * 60)

# 5a. Label encode binary categoricals
le = LabelEncoder()
binary_cols = ['Attrition', 'OverTime', 'Gender']
for col in binary_cols:
    df[col] = le.fit_transform(df[col])
    print(f"    LabelEncoder → {col} (classes: {list(le.classes_)}) → {list(range(len(le.classes_)))}")

print(f"\n    Reason: Binary columns only have 2 values – LabelEncoder")
print(f"    converts them to 0/1 without creating unnecessary columns.")

# 5b. One-Hot Encode multi-class categoricals
ohe_cols = ['Department', 'JobRole', 'MaritalStatus', 'EducationField', 'BusinessTravel']
df = pd.get_dummies(df, columns=ohe_cols, drop_first=True)
print(f"\n    One-Hot Encoding applied to: {ohe_cols}")
print(f"    Reason: Multi-class nominal columns – no ordinal relationship")
print(f"            exists, so integer encoding would imply false ordering.")
print(f"\n    Shape after encoding: {df.shape[0]} rows × {df.shape[1]} columns")
```

[5] ENCODING CATEGORICAL VARIABLES

```
LabelEncoder → Attrition (classes: ['No', 'Yes'] → [0, 1])
LabelEncoder → OverTime (classes: ['No', 'Yes'] → [0, 1])
LabelEncoder → Gender (classes: ['Female', 'Male'] → [0, 1])
```